

Quantum-in-the-Loop Control: MPC Optimization Using QAOA for Robotics

Ojas Mediratta

School of Interactive Computing
Georgia Institute of Technology
Atlanta, GA, USA

Abstract—Model predictive control (MPC) is a widely used control strategy for robotic systems because it can optimize future actions while accounting for dynamics, constraints, and task objectives. However, MPC can become computationally expensive when the control space is discrete or combinatorial, especially as the planning horizon grows. This report explores a quantum-in-the-loop formulation of MPC in which the finite-horizon control problem is converted into a quadratic unconstrained binary optimization (QUBO) problem and approximately solved using the Quantum Approximate Optimization Algorithm (QAOA). A simple one-dimensional robotics test problem is used to illustrate the full pipeline: defining discrete control actions, encoding them as binary variables, adding penalties for constraints, constructing the QUBO objective, and running the resulting problem in Qiskit. The goal is not to show near-term quantum advantage, but to clarify how a robotics MPC problem can be mapped onto a quantum optimization framework and to discuss where such an approach may become meaningful as quantum hardware improves.

I. INTRODUCTION

Robotic systems often need to make control decisions under dynamic constraints, limited sensing, and changing task objectives. Model predictive control (MPC) is attractive in this setting because it repeatedly solves a finite-horizon optimization problem, applies the first control input, observes the new state, and then replans. This receding-horizon structure allows MPC to respond to disturbances while still planning ahead [1].

The main limitation of MPC is computational cost. For many continuous control problems, classical convex or nonlinear optimization methods are effective. However, the problem becomes harder when the robot must choose from a finite set of modes, actions, behaviors, contact states, or actuator combinations. In these cases, the optimization problem can take on a combinatorial structure, where the number of candidate input sequences grows exponentially with the planning horizon.

Recent work has begun exploring whether quantum optimization hardware can be used for finite-input MPC problems. Inoue and Yoshida formulated MPC for finite-input systems as a quadratic unconstrained binary optimization problem and solved it using a D-Wave quantum annealer, showing one example of how receding-horizon control can be mapped onto quantum optimization hardware [2]. While that work used quantum annealing rather than a gate-based variational algorithm, it motivates a similar question: whether discrete

MPC problems can be expressed in a form suitable for other quantum optimization methods.

Quantum optimization algorithms are one possible long-term direction for addressing combinatorial control problems. In particular, the Quantum Approximate Optimization Algorithm (QAOA) is designed to approximately solve discrete optimization problems by encoding a cost function into a quantum circuit and optimizing circuit parameters using a classical outer loop [3], [4]. This makes QAOA naturally compatible with binary optimization formulations such as QUBO [5]. Since many discrete MPC problems can be written using binary decision variables, this creates a possible path for quantum-in-the-loop control.

This report develops a small but complete example of that pipeline. A simplified robotics MPC problem is formulated using discrete acceleration choices. The control decisions are encoded as binary variables, the finite-horizon objective is rewritten as a QUBO, and QAOA is used through Qiskit to search for a low-cost control sequence. The example is intentionally simple so that the conversion process is clear, but the same structure can be extended to richer robotic systems with larger discrete action spaces.

II. MODEL PREDICTIVE CONTROL

MPC solves an optimization problem over a finite planning horizon at each control step. Given the current state x_0 , a dynamics model, and a cost function, MPC chooses a sequence of future controls that minimizes predicted tracking error among other task-dependent penalties. Only the first control input is applied, after which the state is updated and the process repeats. This receding-horizon structure is a standard feature of MPC [1], [6].

For a discrete-time system,

$$x_{k+1} = f(x_k, u_k), \quad (1)$$

where x_k is the state and u_k is the control input at timestep k , a standard finite-horizon MPC problem can be written as

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N), \quad (2)$$

subject to the system dynamics and any state or control constraints. Here, N is the planning horizon, $\ell(x_k, u_k)$ is the stage cost, and $\ell_f(x_N)$ is the terminal cost [6].

In classical robotics applications, MPC is often solved using numerical optimization. If the dynamics are linear and the objective is quadratic with continuous controls, the problem may be solved efficiently as a quadratic program. However, if the input u_k must be selected from a discrete set, the optimizer must search over combinations of actions. If there are m possible actions per timestep and a horizon of length N , then the number of possible action sequences is

$$m^N. \quad (3)$$

This exponential scaling motivates the use of alternative optimization tools for discrete or hybrid MPC problems, especially finite-input MPC formulations [2].

III. QAOA AND BINARY OPTIMIZATION

QAOA is a hybrid quantum-classical algorithm for approximate combinatorial optimization. The optimization problem is encoded into a cost Hamiltonian H_C , whose low-energy states correspond to low-cost binary solutions. A mixer Hamiltonian H_M is then used to move probability amplitude between candidate bitstrings [3], [4]. For depth p , the QAOA circuit prepares the parameterized state

$$|\psi(\gamma, \beta)\rangle = \prod_{i=1}^p e^{-i\beta_i H_M} e^{-i\gamma_i H_C} |+\rangle^{\otimes n}, \quad (4)$$

where γ and β are variational parameters optimized by a classical optimizer.

The classical optimization loop evaluates the expected cost

$$\langle H_C \rangle = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle, \quad (5)$$

and updates γ and β to reduce it. In this sense, QAOA resembles methods like machine learning; the quantum circuit produces a parameterized distribution over candidate solutions, and a classical optimizer adjusts the parameters based on measured cost [3], [4].

A convenient input format for QAOA is a quadratic unconstrained binary optimization problem. A QUBO has the form

$$\min_{z \in \{0,1\}^n} z^T Q z, \quad (6)$$

where z is a vector of binary decision variables and Q is a matrix defining the linear and quadratic cost terms [5]. Because QUBO problems are unconstrained, constraints are usually included as penalty terms inside the objective. This is the key idea that allows a constrained MPC problem to be converted into a form suitable for QAOA.

IV. ROBOTICS TEST PROBLEM

To demonstrate the conversion from MPC to QUBO, this report uses a simple one-dimensional robot model. The robot moves along a line toward a desired goal position, as shown in Fig. 1. At each timestep, the controller selects one of a small number of discrete acceleration actions.

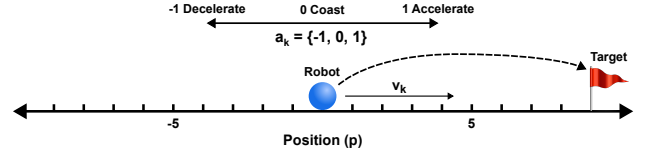


Fig. 1. One-dimensional point-robot test problem used for the MPC-to-QUBO formulation. The robot starts from an initial position and must choose discrete acceleration commands to move toward the target position over a finite prediction horizon.

The robot has position p_k and velocity v_k , so its state is

$$x_k = [p_k \ v_k]^T. \quad (7)$$

At each timestep, the robot chooses one of three discrete acceleration actions:

$$a_k \in \{-a_{\max}, 0, +a_{\max}\}. \quad (8)$$

These correspond to decelerating, coasting, or accelerating. The discrete-time dynamics are

$$p_{k+1} = p_k + \Delta t v_k, \quad (9)$$

$$v_{k+1} = v_k + \Delta t a_k. \quad (10)$$

The MPC objective is to drive the robot toward a desired position p_{ref} while avoiding unnecessary control effort. A representative finite-horizon objective is

$$J = \sum_{k=1}^N q_p (p_k - p_{\text{ref}})^2 + \sum_{k=0}^{N-1} r_a a_k^2, \quad (11)$$

where q_p weights position tracking and r_a weights acceleration effort. Although this example is simple, it captures the important structure here; the robot must choose from discrete actions across a horizon, and the quality of each action sequence is evaluated by a cost function.

V. CONVERTING DISCRETE MPC TO QUBO

The first step is to encode the robot's discrete action choices using binary variables. For each timestep k , three binary variables are introduced:

$$z_k^-, \ z_k^0, \ z_k^+ \in \{0, 1\}. \quad (12)$$

These represent decelerate, coast, and accelerate, respectively. A one-hot constraint enforces that exactly one action is selected at each timestep:

$$z_k^- + z_k^0 + z_k^+ = 1. \quad (13)$$

The acceleration can then be recovered from the binary variables as

$$a_k = -a_{\max} z_k^- + 0 z_k^0 + a_{\max} z_k^+. \quad (14)$$

Because QUBO is unconstrained, the one-hot constraint is folded into the objective using a quadratic penalty, following the common approach of converting constrained binary optimization problems into unconstrained QUBO form [2], [5]:

$$J_{\text{penalty}} = \lambda \sum_{k=0}^{N-1} (z_k^- + z_k^0 + z_k^+ - 1)^2. \quad (15)$$

Here, λ must be chosen large enough that invalid action encodings are more costly than valid alternatives.

Next, the dynamics are unrolled over the horizon so that future positions and velocities are expressed directly in terms of the binary acceleration decisions. Since the acceleration is a linear function of the binary variables, the velocity and position predictions are also linear functions of those variables. Squared tracking errors such as $(p_k - p_{\text{ref}})^2$ therefore become quadratic functions of the binary variables. The control effort term a_k^2 is also quadratic in the same variables.

After expanding the tracking cost, effort cost, and penalty cost, the full MPC objective can be written as

$$J_{\text{QUBO}}(z) = z^T Q z + c, \quad (16)$$

where Q contains the linear and quadratic coefficients and c is a constant offset that does not affect the optimizer. Since QAOA searches over binary strings, the constant term can be ignored during optimization and added back only when reporting the final objective value.

VI. IMPLEMENTATION IN QISKIT

The implementation was written in Python using Qiskit's optimization and algorithm libraries. The code for the project is available in the [project repository](#). The implementation uses a one-dimensional point-robot model with state $x_k = [p_k, v_k]^T$ and a discrete acceleration input set $a_k \in \{-a_{\text{max}}, 0, +a_{\text{max}}\}$. The MPC problem is defined by selecting the horizon N , timestep Δt , acceleration magnitude a_{max} , initial state x_0 , reference position p_{ref} , and cost weights for tracking error, velocity, control effort, and terminal error.

The MPC-to-QUBO conversion is implemented by assigning three binary variables to each timestep, corresponding to decelerate, coast, and accelerate. These variables are constrained using a one-hot condition so that exactly one action is selected at each timestep. The robot dynamics are then unrolled over the prediction horizon so that the predicted positions and velocities are written as affine functions of the binary variables. Since the stage costs and terminal costs are quadratic in position, velocity, and acceleration, expanding these terms produces a quadratic polynomial in the binary variables. The one-hot constraints are also added as quadratic penalty terms, producing an unconstrained QUBO objective suitable for quantum optimization.

The resulting binary optimization problem is solved in two ways. First, an exact classical baseline is computed by brute-force enumeration over all 3^N possible action sequences. This provides a ground-truth optimum for the small test problem. Second, the same QUBO is solved through Qiskit using QAOA. In the QAOA case, the QUBO is converted into a cost operator, and a parameterized quantum circuit is optimized using a classical optimizer. After optimization, sampled bitstrings are decoded back into acceleration sequences, and feasible samples are checked against the one-hot constraints.

The implementation also evaluates the method in a receding-horizon loop. At each MPC step, the optimizer computes a finite-horizon control sequence, but only the first action

is applied to the robot. The state is then updated, and the optimization is solved again from the new state. This makes the simulation closer to standard MPC behavior than a single open-loop trajectory optimization.

VII. RESULTS AND DISCUSSION

The implementation was evaluated using both a single finite-horizon optimization instance and a closed-loop receding-horizon simulation. For the single optimization instance, the brute-force solver provides the ground-truth optimum because it enumerates all 3^N possible action sequences. The exact QUBO solver is used to verify that the QUBO formulation preserves the same optimum after the MPC objective and one-hot constraints are converted into binary form. QAOA is then evaluated as the approximate quantum optimization method.

TABLE I
MEASURED RESULT SUMMARY FOR THE DEMO INSTANCE ($N = 3$). THE QAOA RUN USED $p = 1$, MAXITER=20, AND SEED 7.

Method	Cost	Valid?	Actions	Runtime (s)
Brute force	200.3	Yes	[1, 1, -1]	6.66×10^{-5}
Exact QUBO	200.3	Yes	[1, 1, -1]	9.60×10^{-3}
QAOA	203.2	Yes	[1, 1, 0]	2.79×10^1

As shown in Table I, the exact QUBO solution matched the brute-force MPC solution, producing the same optimal action sequence [1, 1, -1] and the same cost of 200.3. This agreement is important because it validates the MPC-to-QUBO conversion: the binary formulation did not change the optimum of the original finite-input control problem. The QAOA solution also produced a valid one-hot action sequence, but it selected [1, 1, 0] with a slightly higher cost of 203.2. This result is expected for a low-depth QAOA circuit, since QAOA is an approximate variational method and the result depends on the circuit depth, optimizer settings, sampling, and initialization.

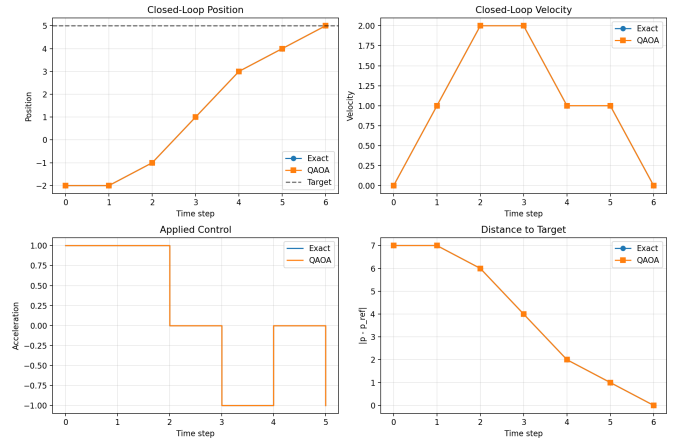


Fig. 2. Closed-loop receding-horizon behavior for the exact QUBO and QAOA controllers. Both methods drive the robot from the initial position to the target, while producing nearly identical position, velocity, applied-control, and distance-to-target profiles in this run.

The closed-loop behavior is shown in Fig. 2. In this simulation, both the exact QUBO and QAOA controllers drive

the robot from the initial position to the target position. The applied control sequence shows the expected structure for this simple point-mass problem: the robot initially accelerates toward the target, then reduces or reverses acceleration as it approaches the desired position. Although the single-step QAOA result in Table I is slightly suboptimal, the receding-horizon simulation still achieves the desired closed-loop behavior because the controller replans after each applied action.

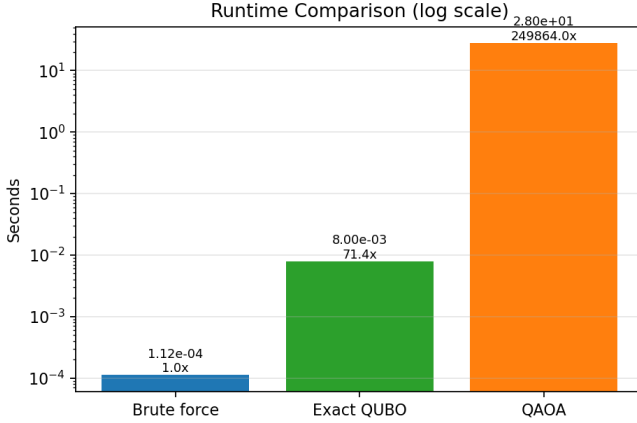


Fig. 3. Runtime comparison for brute-force enumeration, exact QUBO solving, and QAOA on the demo instance. The plot uses a logarithmic scale because QAOA is much slower than the classical baselines for this small problem.

The runtime comparison in Fig. 3 shows the practical limitation of this implementation. Brute-force enumeration was fastest for the small horizon because the search space contained only 3^N action sequences. The exact QUBO method was slower than brute force due to the overhead of constructing and solving the binary optimization problem through Qiskit, but it still completed quickly. QAOA was much slower, taking on the order of tens of seconds for the same small problem. This overhead comes from repeatedly evaluating the parameterized quantum circuit while a classical optimizer updates the QAOA angles.

These results should not be interpreted as evidence of near-term quantum advantage. For a small one-dimensional robot with a short horizon, classical methods are clearly more practical. Instead, the main value of the experiment is that it demonstrates the full pipeline from finite-input MPC to QUBO to QAOA. The approach becomes more interesting for control problems with stronger combinatorial structure, such as mode scheduling, contact planning, multi-robot task allocation, or MPC over a large library of candidate motion primitives. In those cases, the number of candidate control sequences can grow rapidly with the horizon and action set size.

Hardware limitations are also important. In a direct QUBO-to-QAOA mapping, each binary decision variable corresponds to a qubit, so increasing the horizon or the number of candidate actions quickly increases the qubit requirement. QAOA also requires repeated circuit evaluations inside a classical optimization loop, which introduces latency that is not suitable

for fast real-time robotic control on current hardware. For this reason, near-term quantum MPC is more realistic as an offline planning, benchmarking, or simulation tool than as a direct onboard controller. However, the results show that the mathematical mapping is feasible and that QAOA can produce valid control actions for a discrete MPC problem.

VIII. CONCLUSION

This report presented a quantum-in-the-loop formulation of discrete MPC for a simple robotics test problem. The robot’s discrete acceleration choices were encoded using one-hot binary variables, the MPC objective was converted into a QUBO using quadratic penalty terms, and the resulting binary optimization problem was solved using QAOA in Qiskit. The example shows how MPC can be connected to quantum optimization methods, even though current quantum hardware is not yet practical for real-time robotic control.

In ideal future scenarios, this approach may become useful for MPC problems with strongly combinatorial structure, especially when robots must choose among many discrete actions, modes, contacts, or multi-agent assignments. Improvements in fault-tolerant quantum computing, lower-latency quantum-classical integration, and larger reliable qubit counts would be necessary before this type of method could compete with mature classical MPC solvers. Still, the QUBO formulation provides a useful bridge between robotics control and quantum optimization, making it a meaningful framework for exploring future quantum-enabled robotic planning.

REFERENCES

- [1] MathWorks. “What Is Model Predictive Control?” Accessed: 2026-04-25. [Online]. Available: <https://www.mathworks.com/help/mpc/gs/what-is-mpc.html>.
- [2] D. Inoue and H. Yoshida, “Model Predictive Control for Finite Input Systems using the D-Wave Quantum Annealer,” *Scientific Reports*, vol. 10, no. 1, p. 330, 2020. DOI: 10.1038/s41598-020-58081-9. [Online]. Available: <https://doi.org/10.1038/s41598-020-58081-9>.
- [3] E. Farhi, J. Goldstone, and S. Gutmann, *A quantum approximate optimization algorithm*, 2014. arXiv: 1411.4028 [quant-ph]. [Online]. Available: <https://arxiv.org/abs/1411.4028>.
- [4] IBM Quantum. “Quantum Approximate Optimization Algorithm.” Accessed: 2026-04-25, IBM Quantum Documentation. [Online]. Available: <https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm>.
- [5] F. Glover, G. Kochenberger, and Y. Du, *A tutorial on formulating and using qubo models*, 2018. arXiv: 1811.11538 [cs.DS]. [Online]. Available: <https://arxiv.org/abs/1811.11538>.
- [6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.